# Sprint Planner

Sprint Planner is a collaborative Jira-style showcase deployed entirely through ordinary AWS CDK v2 to aws-sim. It provisions DynamoDB, Lambda, API Gateway HTTP and WebSocket APIs, EventBridge, Standard SQS queues and DLQs, SES, CloudWatch, IAM, and a versioned S3 website. Cognito is deliberately external: the stack contains no `AWS::Cognito::*` resource and never owns or deletes the manually configured pool, app client, or users.

The application opens with one deterministic Northstar Product workspace, one active sprint, planning and completed sprint history, fifteen tickets, comments, activity, a pending bootstrap administrator, accessible drag-and-drop, conflict-safe moves, live updates, invitations, and asynchronous notifications.

## aws-sim compatibility note

aws-sim's bounded `AWS::S3::Bucket` CloudFormation provider accepts only `BlockPublicAcls` and `IgnorePublicAcls` inside `PublicAccessBlockConfiguration`. CDK also synthesizes `BlockPublicPolicy` and `RestrictPublicBuckets` when they are explicitly false. The website stack therefore narrows that generated property to the supported pair while retaining a bounded, anonymous, read-only `s3:GetObject` policy. No application behavior is lost.

## Prerequisites

- Node.js 22.13 or newer
- a built, running aws-sim (IAM enforcement is the default)
- the repository's default local AWS credentials ( `test` / `test` are suitable for the usual local bootstrap)
- a manually created same-account, same-Region Cognito email user pool and public app client

### Create the Cognito prerequisites in the aws-sim console

The CDK application deliberately does not create or own Cognito resources. Create the pool and app client before running `npm run deploy` :

1. Open the aws-sim console at `http://127.0.0.1:4566/_aws-sim/console`.
2. Choose **Cognito → User pools → Create user pool**.
3. Configure the pool:
   - **Sign-in option:** `Email address`
   - **Enable self-service sign-up:** checked
   - **Automatically verify email by confirmation code:** checked
   - **Require email:** checked
   - **Usernames are case sensitive: unchecked**
   - **Email delivery:** Cognito default
4. Create the pool, open it, and choose **App clients → Create app client**.
5. Configure the public browser client:
   - **App client name:** `sprint-planner-browser`
   - `ALLOW_USER_PASSWORD_AUTH` : checked
   - `ALLOW_REFRESH_TOKEN_AUTH` : checked
   - **Generate a client secret: unchecked**
   - **Prevent user-existence errors:** checked
   - **Enable token revocation:** checked
   - **Enable refresh-token rotation: unchecked**
   - Access and ID token validity: `1` hour
   - Refresh token validity: `7` days
6. Copy the pool ID and the app client's **Client ID** into `config/local.json`.

Username case sensitivity is fixed when a Cognito pool is created. If `npm run check:cognito` reports `email usernames must be case-insensitive`, create another pool with **Usernames are case sensitive** unchecked. You may retain the old pool; Sprint Planner only uses the pool named in `config/local.json`.

Do not create the bootstrap administrator in Cognito at this stage. After deployment, use the Sprint Planner sign-up screen with the exact `bootstrapAdmin.email` from `config/local.json`, retrieve its confirmation code from the local SES Inbox, confirm the account, and claim administrator access. An administrator-created Cognito user with a temporary password is not required.

# Configure

```
cd examples/cdk-sprint-planner
npm install
cp config/local.example.json config/local.json
```

Edit `config/local.json` with the exact local account, Region, endpoints, manual pool/client identifiers, bootstrap administrator email/display name, and application sender email. This file is ignored. It rejects credentials, passwords, tokens, codes, client secrets, non-loopback endpoints, and unknown fields.

The Cognito portion should contain only public identifiers:

```
"cognito": {
  "userPoolId": "eu-west-1_xxxxxxxxx",
  "appClientId": "paste-the-public-client-id-here"
}
```

Check the manual Cognito boundary:

```
npm run check:cognito
```

# Deploy

```
npm run deploy
```

The first run deploys the data and placeholder website stacks, then the application stack. The SES sender initially remains pending. Open the verification message in the local SES Inbox, verify it, and rerun `npm run deploy`. The resumable run seeds the workspace, publishes final public runtime configuration, redeploys only the website, and exercises the complete outbox → EventBridge → relay → SQS → worker smoke path.

The command writes `.runtime/deployment.json`. It contains only public identifiers and resource names—never credentials, passwords, codes, invitation/WebSocket tickets, or user tokens.

# Work with the showcase

1. Open the printed website URL and create the configured bootstrap administrator Cognito account.

2. Read its six-digit confirmation code in the local SES Inbox and confirm.
3. Sign in and claim administrator access. Exact verified-email matching is required.
4. Open **Team**, invite a second email, and follow its hash-route link.
5. Sign up and confirm the invited user, accept membership, then open both sessions to see live assignment and board moves.
6. As the administrator, open a ticket and choose any active teammate from the **Assignee** list. The current assignee can use the same list to reassign or unassign the ticket.

Administrators can create, edit, assign, plan, archive, and manage sprints/team invitations. Members see the whole shared board, comment, move only their assigned active-sprint tickets, and reassign tickets they currently own. Other members see the assignee without an editable control. Lambda and current DynamoDB membership enforce every rule.

# Verify

```
npm run build
npm test
npm run smoke
SPRINT_PLANNER_ADMIN_PASSWORD='ephemeral-value' \
SPRINT_PLANNER_MEMBER_PASSWORD='another-ephemeral-value' \
  npm run test:collaboration
SPRINT_PLANNER_ADMIN_PASSWORD='ephemeral-value' npm run capture
```

`npm run seed` is idempotent and never overwrites changed or user-created records.
`npm run seed -- --reset-demo` is the explicit bounded reset for known seed-owned data. It refuses unsafe references and never changes Cognito or unrelated application records.

If an outbox record exhausts stream retries:

```
npm run replay:outbox -- --event-id <exact-event-id>
```

The replay command reads the exact pending record, invokes the publisher with a key-only operator envelope, retains the original event identity/body, waits for publication, and removes only its matching failure receipt.

# Destroy

```
npm run destroy
```

Destroy removes the App and Data resources according to their policies. The populated versioned website bucket is retained, as are the manually owned Cognito pool, app client, users, and SES Inbox history. Inspect and empty/delete the retained website bucket separately when you no longer need it.